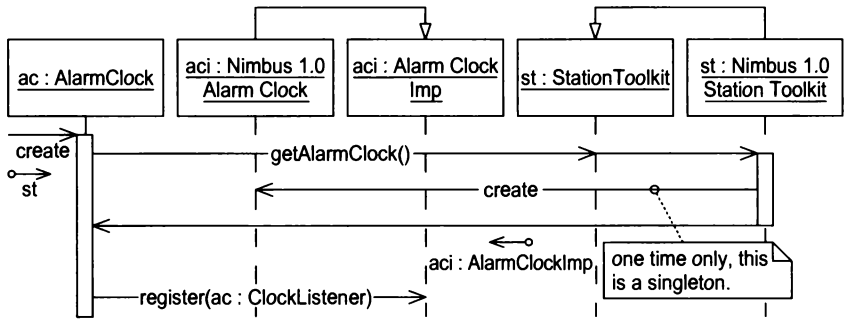


Figure 27-12 Creation of the Alarm Clock

```
    AlarmClock ac = new AlarmClock(st);
    TemperatureSensor ts =
      new TemperatureSensor(ac,st);

    BarometricPressureSensor bps =
      new BarometricPressureSensor(ac,st);

    BarometricPressureTrend bpt =
      new BarometricPressureTrend(bps)
  }
}
```

This is pretty good, but in Java we can do even better. Java allows us to create objects by name. The main program in Listing 27-5 does not need to be changed in order to make it work with a new platform. The name of the `StationToolkit` derivative is simply passed in as a command-line argument. If the name was correctly specified, the appropriate `StationToolkit` will be created, and the rest of the system will behave appropriately.

Listing 27-5

`WeatherStation`

```
public class WeatherStation
{
  public static void main(String[] args)
  {
    try
    {
      Class tkClass = Class.forName(args[0]);
      StationToolkit st =
        (StationToolkit)tkClass.newInstance();

      AlarmClock ac = new AlarmClock(st);

      TemperatureSensor ts =
        new TemperatureSensor(ac,st);

      BarometricPressureSensor bps =
        new BarometricPressureSensor(ac,st);

      BarometricPressureTrend bpt =
        new BarometricPressureTrend(bps)
    }
```

```
    catch (Exception e)
    {
    }
  }
}
```

Putting the Classes into Packages. There are several portions of this software that we would like to release and distribute separately. The API and each of its instantiations are reusable without the rest of the application and may be used by the testing and quality assurance teams. The UI and sensors should be separate so that they can vary indepedently. After all, newer products may have better UI's on top of the same system architecture. In fact, Release II will be the first example of this.

Figure 27-13 shows a package structure for Phase I. This package structure nearly falls out of the classes we have designed so far. There is one package for each platform, and the classes in those packages derive from the classes in the API package. The sole client of the API package is the `WeatherMonitoringSystem` package, which holds all the other classes.

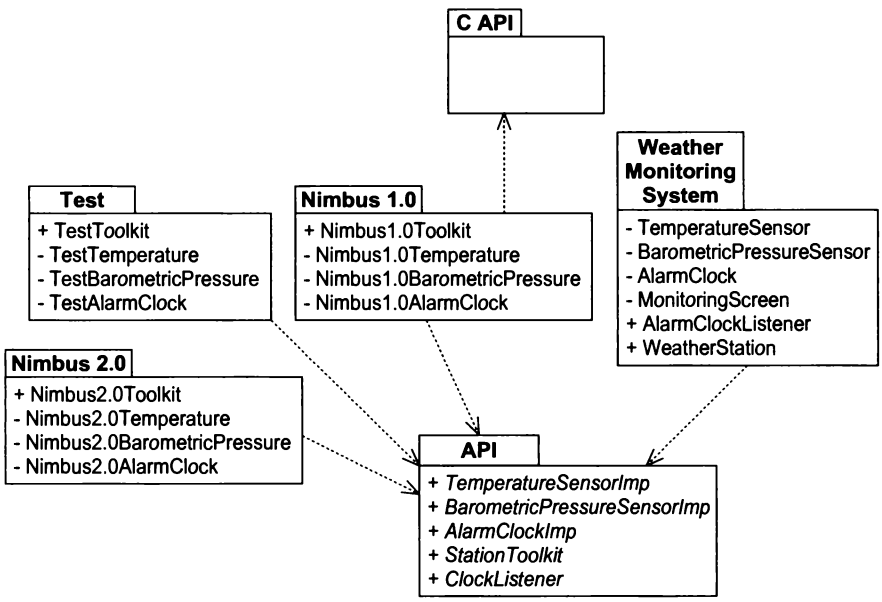


Figure 27-13 Phase I Package Structure

Even though Release I has a very small UI, it is unfortunate that it is mixed in with the `WeatherMonitoringSystem` classes. It would be better to put this class in a separate package. However, we have a problem. As things stand, the `WeatherStation` object creates the `MonitoringScreen` object, but the `MonitoringScreen` object must know about all the sensors in order to add its observers through their `Observable` interface. Thus, if we were to pull the `MonitoringScreen` out into its own package, there would be a cyclic dependency between that package and the `WeatherMonitoringSystem` package. This violates the acyclic-dependencies principle (ADP) and would make the two packages impossible to release independently of each other.

We can fix this by pulling the main program out of the `WeatherStation` class. `WeatherStation` still creates the `StationToolkit` and all the sensors, but it does not create the `MonitoringScreen`. The main program will create the `MonitoringScreen` and the `WeatherStation`. The main program will then pass the `WeatherStation` to the `MonitoringScreen` so that the `MonitoringScreen` can add its observers to the sensors.

How does the MonitoringScreen get the sensors from the WeatherStation? We need to add some methods to the WeatherStation that allow this to take place. See Listing 27-6 to see what this looks like.

Listing 27-6

WeatherStation

```
public class WeatherStation
{
  public WeatherStation(String tkName)
  {
    //create station toolkit and sensors as before.
  }

  public void addTempObserver(Observer o)
  {
    itsTS.addObserver(o);
  }

  public void addBPObserver(Observer o)
  {
    itsBPS.addObserver(o);
  }

  public void addBPTrendObserver(Observer o)
  {
    itsBPT.addObserver(o);
  }

// private variables...
  private TemperatuerSensor itsTS;
  private BarometricPressureSensor itsBPS;
  private BarometricPressureTrend itsBPT;
}
```

Now we can redraw the package diagram as shown in Figure 27-14. We have omitted most of the packages that aren't concerned with the MonitoringScreen. This looks pretty good. Certainly the UI can be varied without affecting the WeatherMonitoringSystem. However, the dependency of the UI on WeatherMonitoringSystem will cause problems whenever the WeatherMonitoringSystem changes.

Both UI and WeatherMonitoringSystem are concrete. When one concrete package depends on another, the dependency inversion principle (DIP) is violated. In this case, it would be better if the UI depended on something abstract rather than the WeatherMonitoringSystem.

We can fix this by creating an interface that the MonitoringScreen can use and that the WeatherStation derives from. (See Figure 27-15.)

Now, if we put the WeatherStationComponent interface into its own package, we will achieve the separation we want. (See Figure 27-16.) Notice that now the UI and the WeatherMonitoringSystem are completely decoupled. They can both vary independently of each other. This is a good thing.

24-Hour History and Persistence

Points four and five of the Release I deliverables section (see page 382) talk about the need for maintaining a persistent 24-hour history. We know that both the Nimbus 1.0 and Nimbus 2.0 hardware have some kind of nonvolatile memory (NVRAM). On the other hand, the test platform will simulate the nonvolatile memory by using the disk.

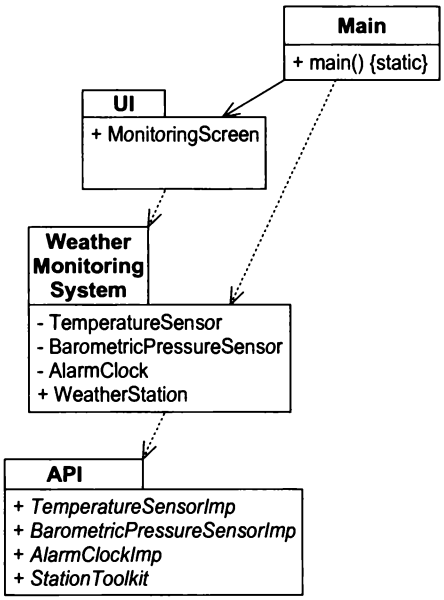


Figure 27-14 Package Diagram with Cycle Broken

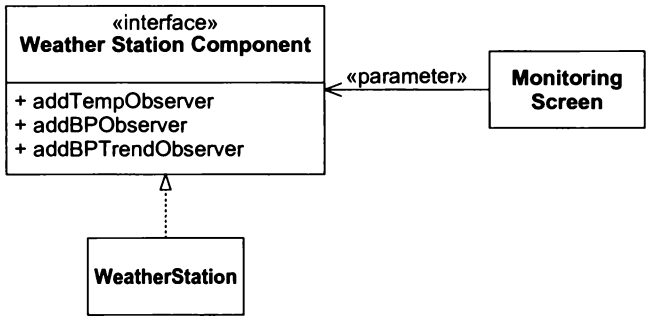


Figure 27-15 WeatherStation abstract interface

We need to create a persistence mechanism that is independent of the individual platforms, while still providing the necessary functionality. We also need to connect this to the mechanisms that maintain the 24-hour historical data.

Clearly, the low-level persistence mechanism should be defined as an interface in the API package. What form should this interface take? The Nimbus I C-API provides calls that allow blocks of bytes to be read and written from particular offsets within the nonvolatile memory. While this is effective, it is also somewhat primitive. Is there a better way?

The Persistent API. The Java environment provides the facilities to allow any object to be immediately converted into an array of bytes. This process is called *serialization*. Such an array of bytes can be reconstituted back into an object through the process of *deserialization*. It would be convenient if our low-level API allowed us to specify an object and a name for that object. Listing 27-7 shows what this might look like.

Listing 27-7

`PersistentImp`

```
package api;
import java.io.Serializable;
```

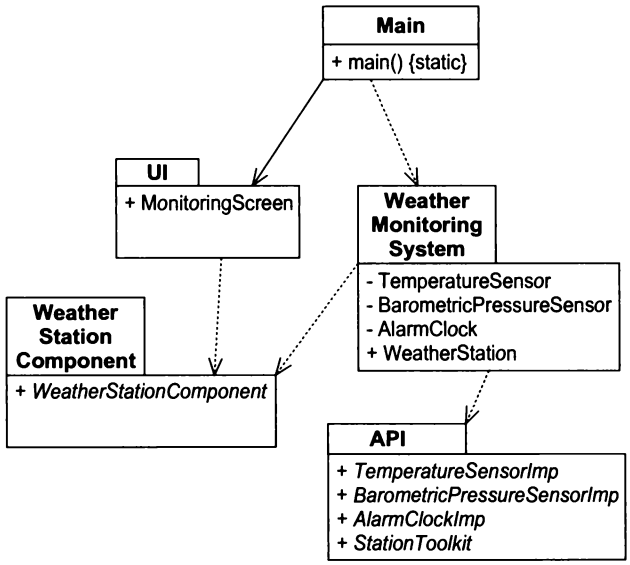


Figure 27-16 Weather Station Component Package Diagram

```
import java.util.AbstractList;
public interface PersistentImp
{
  void store(String name, Serializable obj);
  Object retrieve(String name);
  AbstractList directory(String regExp);
};
```

The `PersistentImp` interface allows you to `store` and `retrieve` full objects by name. The only restriction is that such objects must implement the `Serializable` interface, a very minimal restriction.

24-Hour History. Having decided on the low-level mechanism for storing persistent data, let's look at the kind of data that will be persistent. Our spec says that we must keep a record of the high and low readings for the previous 24-hour period. Figure 27-23 on page 380 shows a graph with these data. This graph does not seem to make a lot of sense. The high and low readings are painfully redundant. Worse, they come from the last 24-hours on the clock and not from the previous calendar day. Usually, when we want the last 24-hour high and low reading, we want it for the previous calendar day.

Is this a flaw in the spec or a flaw in our interpretation? It will do us no good to implement something according to the spec if the spec is not really what the customer wants.

A quick verification with the stakeholders shows our intuition to be correct. We do indeed want to keep a rolling history of the last 24 hours. However, the historical low and high need to be for the previous calendar day.

The 24-hour High and Low. The daily high and low values will be based upon real-time readings of the sensors. For example, every time the temperature changes, the 24-hour high and low temperatures will be updated appropriately. Clearly, this is an OBSERVER relationship. Figure 27-17 shows the static structure, and Figure 27-18 shows the relevant dynamic scenarios.

We have chosen to show the OBSERVER pattern using an association marked with the «observes» stereotype. We have created a class called `TemperatureHiLo` that is woken up by the `AlarmClock` every day at midnight. Notice that the `wakeEveryDay` method has been added to `AlarmClock`.

Upon construction of the `TemperatureHiLo` object, it registers with both the `AlarmClock` and with the `TemperatureSensor`. Whenever the temperature changes, the `TemperatureHiLo` object is notified through the OBSERVER pattern. `TemperatureHiLo` then informs the `HiLoData` interface using the `currentReading`

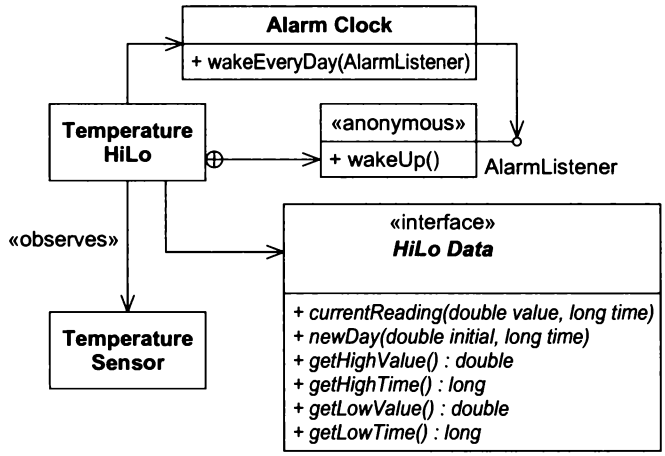


Figure 27-17 `TemperatureHiLo` structure

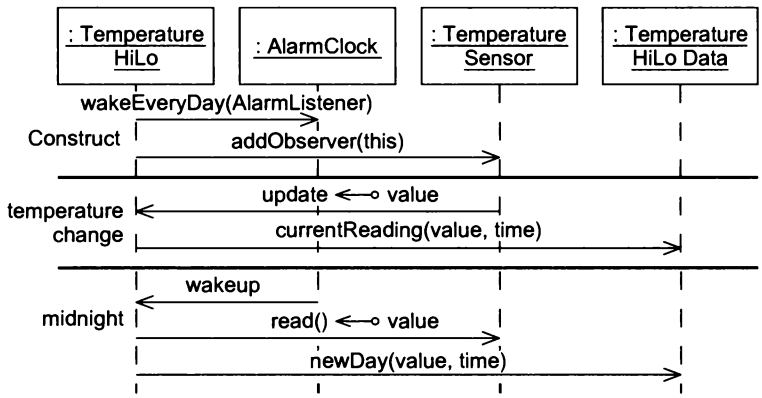


Figure 27-18 HiLo Scenarios

method. `HiLoData` will have to be implemented with some class that knows how to store the high and low values for the current 24-hour calendar day.

We have separated the `TemperatureHiLo` class from the `HiLoData` class for two reasons. First of all, we wanted to separate the knowledge of the `TemperatureSensor` and `AlarmClock` from the algorithms that determined the daily highs and lows. Second, and more importantly, the algorithm for determining the daily highs and lows can be reused for barometric pressure, wind speed, dew point, etc. Thus, though we will need `BarometricPressureHiLo`, `DewPointHiLo`, `WindSpeedHiLo`, etc. to observe the appropriate sensors, each will be able to use the `HiLoData` class to compute and store the data.

At midnight, the `AlarmClock` sends the `wakeup` message to the `TemperatureHiLo` object. `TemperatureHiLo` responds by fetching the current temperature from the `TemperatureSensor` and forwarding it to the `HiLoData` interface. The implementation of `HiLoData` will have to store the previous calendar day's values using the `PersistentImp` interface and will also have to create a new calendar day with the initial value.

`PersistentImp` accesses objects in the persistent store using a string. This string acts as an access key. Our `HiLoData` objects will be stored and retrieved with strings that have the following format: "`<type>+HiLo+<MM><dd><yyyy>`." For example, "`temperatureHiLo04161998`."

Implementing the HiLo Algorithms

How do we implement the `HiLoData` class? This seems pretty straightforward. Listing 27-8 shows what the Java code for this class looks like.

Listing 27-8

HiLoDataImp

```
public class HiLoDataImp implements HiLoData,java.io.Serializable
{
  public HiLoDataImp(StationToolkit st, String type,
                     Date theDate, double init,
                     long initTime)
  {
    itsPI = st.getPersistentImp();
    itsType = type;
    itsStorageKey = calculateStorageKey(theDate);
    try
    {
      HiLoData t =(HiLoData)itsPI.retrieve(itsStorageKey);
      itsHighTime =  t.getHighTime();
      itsLowTime =   t.getLowTime();
      itsHighValue = t.getHighValue();
      itsLowValue =  t.getLowValue();
      currentReading(init, initTime);
    }
    catch (RetrieveException re)
    {
      itsHighValue = itsLowValue = init;
      itsHighTime = itsLowTime = initTime;
    }
  }

  public long   getHighTime()  {return itsHighTime;}
  public double getHighValue() {return itsHighValue;}
  public long   getLowTime()   {return itsLowTime;}
  public double getLowValue()  {return itsLowValue;}

  // Determine if a new reading changes the
  // hi and lo and return true if reading changed.
  public void currentReading(double current, long time)
  {
    if (current > itsHighValue)
    {
      itsHighValue = current;
      itsHighTime = time;
      store();
    }
    else if (current < itsLowValue)
    {
      itsLowValue = current;
      itsLowTime = time;
      store();
    }
  }
```

```
  public void newDay(double initial, long time)
  {
    store();
    // now clear it out and generate a new key.
    itsLowValue = itsHighValue = intial;
    itsLowTime = itsHighTime = time;
    // now calculate a new storage key based on
    // the current date, and store the new record.
    itsStorageKey = calculateStorageKey(new Date());
    store()
  }

  private store()
  {
    try
    {
      itsPI.store(itsStorageKey, this);
    }
    catch (StoreException)
    {
      // log the error somehow.
    }
  }

  private String calculateStorageKey(Date d)
  {
    SimpleDateFormat df = new SimpleDateFormat("MMddyyyy");
    return(itsType + "HiLo" + df.format(d));
  }
  private double itsLowValue;
  private long   itsLowTime;
  private double itsHightValue;
  private long   itsHighTime;
  private String itsType;
  // we don't want to store the following.
  transient private String itsStorageKey;
  transient private api.PersistentImp itsPI;
}
```

Well, maybe it wasn't all *that* straightforward. Let's walk through this code to see what it does.

At the bottom of the class, you'll see the private member variables. The first four variables are expected. They record the high and low values and the times at which those values occurred. The `itsType` variable remembers the type of readings that this `HiLoData` is keeping. This variable will have the value "`Temp`" for temperature, "`BP`" for barometric pressure, "`DP`" for dew point, etc. The last two variables are declared transient. This means that they will not be stored in the persistent memory. They record the current storage key and a reference to the `PersistentImp`.

The constructor takes five arguments. The `StationToolkit` is needed to gain access to the `PersistentImp`. The `type` and `Date` arguments will be used to build the storage key used for storing and retrieving the object. Finally, the `init` and `initTime` arguments are used to initialize the object in the event that `PersistentImp` cannot find the storage key.

The constructor tries to fetch the data from `PersistentImp`. If the data are present, it copies the nontransient data into its own member variables. Then it calls `currentReading` with the initial value and time to make

sure that these readings get recorded. Finally, if `currentReading` discovers that there was a change in the high or low data, it returns true, and the `Store` function is invoked to make sure that the persistent memory is updated.

The `currentReading` method is the heart of this class. It compares the old high and low values with the new incoming reading. If the new reading is higher than the old high or lower than the old low, it replaces the appropriate value, records the appropriate time, and stores the changes in persistent memory.

The `newDay` method is invoked at midnight. First, it stores the current `HiLoData` in persistent memory. Then it resets the values of the `HiLoData` for the beginning of a new day. It recomputes the storage key for the new date and then stores the new `HiLoData` in persistent memory.

The `Store` function simply uses the current storage key to write the `HiLoData` object into persistent memory through the `PersistentImp` object.

Finally, the `calculateStorageKey` method builds a storage key from the type of the `HiLoData` and the date argument.

Ugliness. Certainly the code in Listing 27-8 is not too difficult to understand. However, there is ugliness for another reason. The policy embodied in the functions `currentReading` and `newDay` have to do with managing the high-low data and are independent of persistence. On the other hand, the `store`, and `calculateStorageKey` methods, the constructor, and the `transient` variables are all specific to persistence and have nothing to do with the management of the highs and lows. This is a violation of the SRP.

In its current commingled state, this class has the makings of a maintenance nightmare. If something fundamental about the persistence mechanism changes, to the extent that the `calculateStorageKey` and `store` functions become inappropriate, then new persistence facilities will have to be grafted into the class. Functions like `newDay` and `currentReading` will have to be altered to invoke the new persistence facilities.

Decoupling Persistence from Policy. We can avoid these potential problems by decoupling the high-low data-management policy from the persistence mechanism using the PROXY pattern. Look back at Figure 26-7 on page 346. Note the decoupling of the policy layer (application) from the mechanism layer (API).

Figure 27-19 employs the PROXY pattern to effect the necessary decoupling. It differs from Figure 27-17 on page 371 by the addition of the `HiLoDataProxy` class. It is the proxy class to which the `TemperatureHiLo` object actually holds a reference. The proxy, in turn, holds a reference to a `HiLoDataImp` object and delegates calls to it. Listing 27-9 shows the implementing of the critical functions of both `HiLoDataProxy` and `HiLoDataImp`.

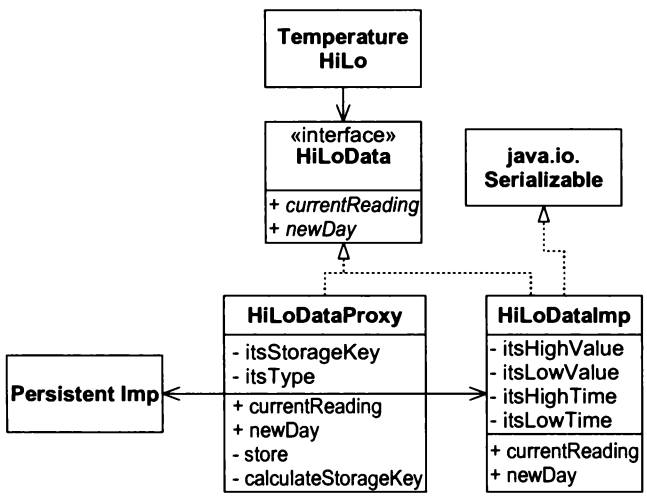


Figure 27-19 Proxy pattern applied to HiLo persistence

Listing 27-9

Snippets of the Proxy solution

```
class HiLoDataProxy implements HiLoData
{
  public boolean currentReading(double current, long time)
  {
    boolean change;
    change = itsImp.currentReading(current, time);
    if (change)
      store();
    return change;
  }

  public void newDay(double initial, long time)
  {
    store();
    itsImp.newDay(initial, time);
    calculateStorageKey(new Date(time));
    store();
  }

  private HiLoDataImp itsImp;
}

class HiLoDataImp implements HiLoData, java.io.Serializable
{
  public boolean currentReading(double current, long time)
  {
    boolean changed = false;
    if (current > itsHighValue)
    {
      itsHighValue = current;
      itsHighTime = time;
      changed = true;
    }
    else if (current < itsLowValue)
    {
      itsLowValue = current;
      itsLowTime = time;
      changed = true;
    }
    return changed;
  }

  public void newDay(double initial, long time)
  {
    itsHighTime = itsLowTime = time;
    itsHighValue = itsLowValue = initial;
  }
};
```

Notice how the HiLoDataImp class has no inkling of persistence. Notice also that the HiLoDataProxy class takes care of all the persistence ugliness and then delegates to the HiLoDataImp. This is nice. Furthermore, notice how the proxy depends on both HiLoDataImp (the policy layer) and PersistentImp (the mechanism layer). This is exactly what we were after.

But all is not perfect. The astute reader will have caught the change that we made to the currentReading method. We changed it to return a boolean. We need this boolean in the proxy so that the proxy knows when to call store. Why don't we call store every time currentReading is called? There are many varieties of NVRAM. Some of them have an upper limit on the number of times you can write to them. Therefore, in order to prolong the life of the NVRAM, we only store into it when the values change. Real life intrudes, yet again.

Factories and Initialization. Clearly, we don't want TemperatureHiLo to know anything about the proxy. It should know only about HiLoData. (See Figure 27-19.) Yet somebody is going to have to create the HiLoDataProxy for the TemperatureHiLo object to use. Also, someone is going to have to create the HiLoDataImp to which the proxy delegates.

What we need is a way to create objects without knowing exactly what type of object we are creating. We need a way for TemperatureHiLo to create a HiLoData without knowing that it is really creating a HiLoDataProxy and a HiLoDataImp. Again, we fall back on the FACTORY pattern. (See Figure 27-20.)

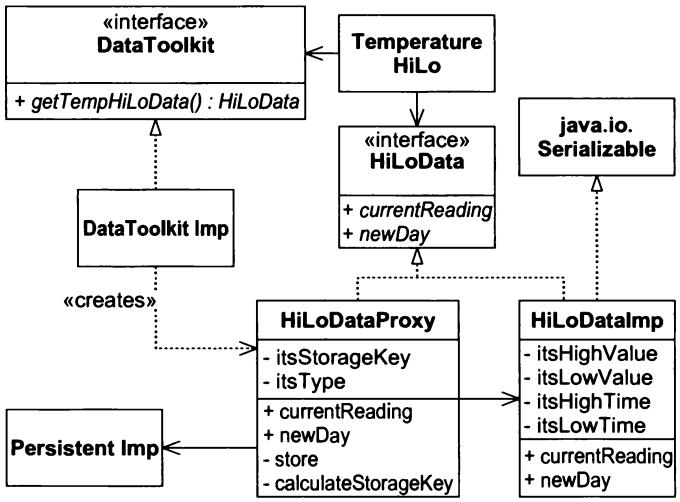


Figure 27-20 Using Abstract Factory to create the Proxy

TemperatureHiLo uses the DataToolkit interface to create an object that conforms to the HiLoData interface. The getTempHiLoData method gets deployed to a DataToolkitImp object, which creates a HiLoDataProxy whose type code is "Temp" and returns it as a HiLoData.

This solves the creation problem nicely. TemperatureHiLo does not need to depend upon the HiLoDataProxy in order to create it. But how does TemperatureHiLo gain access to the DataToolkitImp object? We don't want TemperatureHiLo to know anything about DataTookitImp because that would create a dependency from the policy layer to the mechanism layer.

Package Structure. To answer this question, let's look at the package structure in Figure 27-21. The abbreviation WMS stands for the Weather Monitoring System package that was described in Figure 27-16 on page 370.

Figure 27-21 reenforces our desire for the persistence-interface layer to depend on the policy and mechanism layers. It also shows how we have deployed the classes into the packages. Notice that the abstract factory,

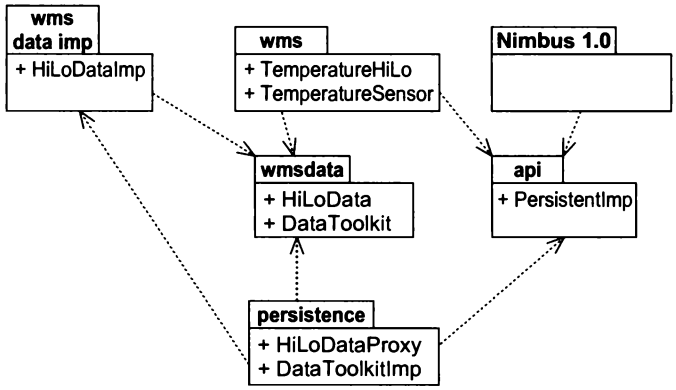


Figure 27-21 PROXY and FACTORY package structure

`DataToolkit`, is defined in the `WMSData` package along with `HiLoData`. `HiLoData` is implemented in the `WMSDataImp` package, whereas `DataToolkit` is implemented in the `persistence` package.

Who Creates the Factory? Now, we ask the question once again. How does the instance of `wms.TemperatureHiLo` gain access to an instance of `persistence.DataToolkitImp` so that it can call the `getTempHiLoData` method and create instances of `persistence.HiLoDataProxy`?

What we need is some statically allocated variable, accessible to the classes in `wmsdata`, that is declared to hold a `wmsdata.DataToolkit`, but which is initialized to hold a `persistence.DataToolkitImp`. Since all variables in Java, including static variables, must be declared in some kind of class, we can create a class named `Scope` that will have the static variables that we need. We will put this class in the `wmsdata` package.

Listings 27-10 and 27-11 show how this works. The `Scope` class in `wmsdata` declares a static member variable that holds a `DataToolkit` reference. The `Scope` class in the `persistence` package declares an `init()` function that creates a `DataToolkitImp` instance and stores it in the `wmsdata.Scope.itsDataToolkit` variable.

Listing 27-10

`wmsdata.Scope`

```
package wmsdata;

public class Scope
{
  public static DataToolkit itsDataToolkit;
}
```

Listing 27-11

`persistence.Scope`

```
package persistence;

public class Scope
{
  public static void init()
  {
    wmsdata.Scope.itsDataToolkit =
                      new DataToolkit();
  }
}
```

There is an interesting symmetry between the packages and the `scope` classes. All the classes in the `wmsdata` package, other than `Scope`, are interfaces that have abstract methods and no variables. But the `wmsdata.Scope` class has a variable and no functions. On the other hand, all the classes in the `persistence` package, other than `Scope`, are concrete classes that have variables. But `persistence.Scope` has a function and no variables.

Figure 27-22 shows how this might be depicted in a class diagram. The `Scope` classes are «utility» classes. All the members of such classes, whether variables or functions, are static—a final element to the symmetry. It would appear that packages which contain abstract interfaces tend to contain utilities that have data and no functions, whereas packages which contain concrete classes tend to contain utilities that have functions and no data.

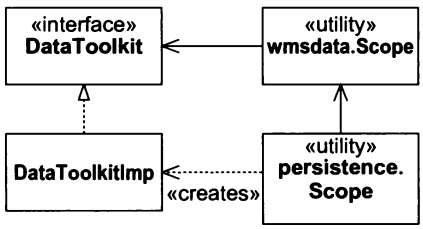


Figure 27-22 Scope Utilities

So, Who Calls `persistence.Scope.init()`? Probably the `main()` function. The class that holds that main function must be in a package that does not mind a dependency upon `persistence`. We often call the package that contains main the `root` package.

But You Said... The persistence-implementation layer should not depend on the policy layer. However, a close inspection of Figure 27-21 shows a dependency from the `persistence` to `wmsDataImp`. This dependency can be traced back to Figure 27-20, in which `HiLoDataProxy` depends on `HiLoDataImp`. The reason for this dependency is so `HiLoDataProxy` can create the `HiLoDataImp` that it depends on.

In most cases, the proxy will not have to create the imp because the proxy will be reading the imp from persistent store. That is, the `HiLoDataImp` will be returned to the proxy by a call to `PersistentImp.retrieve`. However, in those rare cases where the retrieve function does not find an object in the persistent store, `HiLoDataProxy` is going to have to create an empty `HiLoDataImp`.

So, it looks like we need another factory that knows how to create `HiLoDataImp` instances and that the proxy can call. This means more packages and more `Scope` classes, etc.

Is This Really Necessary? Probably not in this case. We created the factory for the proxy because we wanted `TemperatureHiLo` to be able to work with many different persistence mechanisms. Thus, we had a solid benefit to justify the `DataToolkit` factory. But what benefit would be obtained from interposing a factory between `HiLoDataProxy` and `HiLoDataImp`? If there could be many different implementations of `HiLoDataImp`, and if we wanted the proxy to work with them all, then we might be justified.

However, we don't believe that the requirements are quite that volatile. The `wmsDataImp` package contains weather-monitoring policies and business rules that have remained unchanged for quite a while. It seems unlikely that they will be changing any time in the future. This may sound like famous last words, but you have to draw the line somewhere. In this case, we have decided that the dependency from the proxy to the imp does not represent a big maintenance risk, and we will live without the factory.

Conclusion

Jim Newkirk and I wrote this chapter in early 1998. Jim did the bulk of the coding, and I translated the code into UML diagrams and put words around them. The code is now long-gone. But it was the production of that code that drove the design you see in these pages. Most of the diagrams were produced after the code was complete.

In 1998 neither Jim nor I had heard of Extreme Programming. So the design you see here was not conducted in an environment of Pair-programming and test-driven development. However, Jim and I have always worked in a highly collaborative fashion. Together we would pore over the code he wrote, running it where feasible, making design changes together, and then producing the UML and words in this chapter.

So, although the design here is pre-XP, it was still created in a highly collaborative, code-centric way.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Meyer, Bertrand. *Object-Oriented Software Construction*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1997.
3. Arnold, Ken, and James Gosling. *The Java Programming Language*, 2nd ed. Reading, MA: Addison–Wesley, 1998.

Nimbus-LC Requirements Overview

Usage Requirements

This system shall provide automatic monitoring of various weather conditions. Specifically, it must measure the following variables:

- Wind speed and direction
- Temperature
- Barometric pressure
- Relative humidity
- Wind chill
- Dew-point temperature

The system shall also provide an indication of the current trend in the barometric-pressure reading. The three possible values include stable, rising, and falling. For example, the current barometric pressure is 29.95 inches of mercury (IOM) and falling.

The system shall have a display, which continuously indicates all measurements, as well as the current time and date.

24-Hour History

Through the use of a touch screen, the user may direct the system to display the 24-hour history of any of the following measurements:

- Temperature
- Barometric pressure
- Relative humidity

This history shall be presented to the user in the form a line chart. (See Figure 27-23.)

User Setup

The system shall provide the following facilities to the user to allow the station to be configured during installation:

- Setting the current time, date, and time zone.
- Setting the units that will be displayed (English or metric)

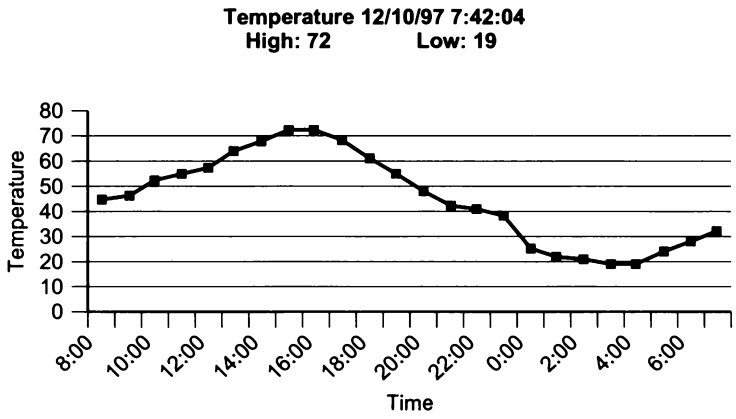


Figure 27-23 Temperature History

Administrative Requirements

The system shall provide a security mechanism for access to the administrative functions of the weather station. These functions include the following:

- Calibrating the sensors against known values
- Resetting the station

Nimbus-LC Use Cases

Actors

In this system, there are two distinct roles played by users.

User. Users view the real-time weather information that the station is measuring. They also interact with the system to display the historical data associated with the individual sensors.

Administrator. The administrator manages the security aspects of the system, calibrating the individual sensors, setting the time/date, setting units of measure, and resetting the station when required.

Use Cases

Use Case #1: Monitor Weather Data. The system will display the current temperature, barometric pressure, relative humidity, wind speed, wind direction, wind chill temperature, dew point, and barometric-pressure trend.

Measurement History

The system will display a line graph depicting the previous 24 hours of readings from the sensors in the system. In addition to the graph, the system will display the current time and date and the highest and lowest readings from the previous 24 hours.

Use Case #2: View Temperature History.

User Case #3: View Barometric Pressure History.

Use Case #4: View Relative Humidity History.

Setup

Use Case #5: Set Units. The user sets the type of units that will be displayed. The choices are between English and metric values. The default is metric.

Use Case #6: Set Date. The user will set the current date.

Use Case #7: Set Time. The user will set the current time and time zone for the system.

Administration

Use Case #8: Reset Weather Station. The administrator has the ability to reset the station back to its factory default settings. It is important to note that this will erase all of the history that is stored in the station and remove any calibration that may have occurred. As one last check, it will inform the administrator of the consequences and prompt for a go/no go to reset the station.

Use Case #9: Calibrate Temperature Sensor. The administrator, using a known good source for the temperature, will enter that value into the system. The system shall accept the value and use it internally to calibrate that actual reading with the readings it is currently measuring. For a detailed look at calibrating the sensors, see the hardware description document.

Use Case #10: Calibrate Barometric-Pressure Sensor.

Use Case #11: Calibrate Relative-Humidity Sensor.

Use Case #12: Calibrate Wind-Speed Sensor.

Use Case #13: Calibrate Wind-Direction Sensor.

Use Case #14: Calibrate Dew-Point Sensor.

Use Case #15: Calibration Log. The system will show the administrator the calibration history of the unit. This history includes the time and date of the calibration, the sensor calibrated, and the value that was used to calibrate the sensor.

Nimbus-LC Release Plan

Introduction

The implementation of the weather station will be done in a series of iterations. Each iteration will build on what has been done previously until we have provided the functionality which is required for release to the customer. This document outlines three releases for this project.

Release I

The release has two goals. The first is to create an architecture that will support the bulk of the application in a manner that is independent of the Nimbus hardware platform. The second goal is to manage the two biggest risks:

1. Getting the old Nimbus 1.0 API to work on the processor board with a new operating system. This is certainly doable, but it is very hard to estimate how long this will take because we cannot anticipate all the incompatibilities.
2. The Java Virtual Machine. We have never used a JVM on an embedded board before. We don't know if it will work with our operating system, or even if it correctly implements all of the Java byte codes properly. Our suppliers assure us that everything will be fine, but we still perceive a significant risk.

The integration of the JVM with the touch screen and graphics subsystem is proceeding in parallel with this release. It is expected to be complete prior to the beginning of the second phase.

Risks

1. Operating system upgrade—We currently use an older version of this OS on our board. In order to use the JVM, we need to upgrade to the latest version of the OS. This also requires us to use the latest version of the development tools.
2. The OS vendor is providing the latest version of the JVM on this version of the OS. In order to stay current, we want to use the 1.2 version of the JVM. However, V1.2 is currently in beta and will change during the construction of the project.
3. Java native interface to the board level "C" API needs to be verified in the new architecture.

Deliverable(s)

1. Our hardware running the new OS along with the latest version of the JVM.
2. A streaming output, which will display the current temperature and barometric-pressure readings. *(Throw away code not used in final release.)*
3. When there is a change in the barometric pressure, the system will inform us as to whether the pressure is rising, falling, or stable.
4. Every hour, the system will display the past 24 hours of measurements for the temperature and barometric pressure. These data will be persistent in that we can cycle the power on the unit and the data will be saved.
5. Every day at 12:00 A.M., the system will display the high and low temperature and barometric pressure for the previous day.
6. All measurements will be in the metric system.

Release II

During this phase of the project, the basis for the user interface is added to the first release. No additional measurements are added. The only change to the measurements themselves is the addition of the calibration mechanism. The primary focus in this phase is on the presentation of the system. The major risk is the software interface to the LCD panel/touch screen. Also, since this is the first release that will display the UI in a form that can be shown to the user, we may begin to have some churn in the requirements. In addition to the software, we will be delivering a specification for the new hardware. This is the main reason for adding of the calibration to this phase of the project. This API will be specified in Java.

Use Cases Implemented

- #2—View Temperature History
- #3—View Barometric-Pressure History
- #5—Set Units
- #6—Set Date
- #7—Set Time/Time Zone
- #9—Calibrate Temperature Sensor
- #10—Calibrate Barometric-Pressure Sensor

Risks

1. The LCD-panel/touch-screen interface to the Java virtual machine needs to be tested on the actual hardware.
2. Requirements changes.
3. Changes in the JVM, along with changes in the Java foundation classes as they proceed from beta to released form.

Deliverable(s)

1. A system that executes and provides all of the functionality specified in the use cases listed above.
2. The temperature, barometric pressure, and time/date portion of Use Case #1 will also be implemented.
3. The GUI portion of the software architecture will be completed as part of this phase.
4. The administrative portion of the software will be implemented to support the temperature and barometric pressure calibrations.
5. A specification for the new hardware API specified in Java instead of "C."

Release III

This is the release prior to customer deployment of the product.

Use Cases Implemented

- #1—Monitor Weather Data
- #4—View Relative-Humidity History
- #8—Reset Weather Station
- #11—Calibrate Relative-Humidity Sensor
- #12—Calibrate Wind-Speed Sensor
- #13—Calibrate Wind-Direction Sensor
- #14—Calibrate Dew-Point Sensor
- #15—Calibration Log

Risks

1. Requirements changes—It is expected that, as more of the product is completed, there may be changes required.
2. Completing the entire product may indicate changes to the hardware API that was specified at the end of Release II.
3. Limits of hardware—As we complete the product, we may run into limitations of the hardware (i.e., memory, CPU, etc.).

Deliverable(s)

1. The new software running on the old hardware platform.
2. A specification for the new hardware that has been validated with this implementation.

SECTION 6

The ETS Case Study

To become a licensed architect in the United States or Canada, you must pass an examination. If you pass, a state licensing board will give you a license to practice architecture. The examination was developed by the Educational Testing Service (ETS) under a charter by the National Council of Architectural Registration Boards (NCARB), and is currently administered by the Chauncey Group International.

In the past, the candidates completed the examination in pencil and paper. These completed examinations were then given to a cadre of jurors for scoring. These jurors were highly experienced architects who would pore over the examinations and decide whether to pass or fail them.

In 1989 NCARB commissioned ETS to research whether or not an automated system could deliver and score portions of the exam. The chapters in this section describe portions of the resulting project. As before, we will encounter a number of useful design patterns in design of this software, so the chapters describing those patterns precede the case study.

28

VISITOR



"'T is some visitor," I muttered, "tapping at my chamber door;
Only this and nothing more."

—Edgar Allen Poe, The Raven

Problem: *You need to add a new method to a hierarchy of classes, but the act of adding it will be painful or damaging to the design.*

This is a common problem. For example, suppose you have a hierarchy of `Modem` objects. The base class has the generic methods common to all modems. The derivatives represent the drivers for many different modem manufacturers and types. Suppose also that you have a requirement to add a new method, named `configureForUnix`, to the hierarchy. This method will configure the modem to work with the UNIX operating system. It will do something different in each modem derivative, because each different modem has its own particular idiosyncrasies for setting its configuration and dealing with UNIX.

Unfortunately, adding `configureForUnix` begs a terrible set of questions. What about Windows? What about MacOs? What about Linux? Must we really add a new method to the `Modem` hierarchy for every new operating system that we use? Clearly this is ugly. We'll never be able to close the `Modem` interface. Every time a new operating system comes along, we'll have to change that interface and redeploy all the modem software.

The VISITOR Family of Design Patterns

The VISITOR family allows new methods to be added to existing hierarchies without modifying the hierarchies.

The patterns in this family are as follows:

- VISITOR
- ACYCLIC VISITOR
- DECORATOR
- EXTENSION OBJECT

VISITOR[1]

Consider the `Modem` hierarchy in Figure 28-1. The `Modem` interface contains the generic methods that all modems can implement. There are three derivatives shown—one that drives a Hayes modem, another that drives a Zoom modem, and a third that drives the modem card produced by Ernie, one of our hardware engineers.

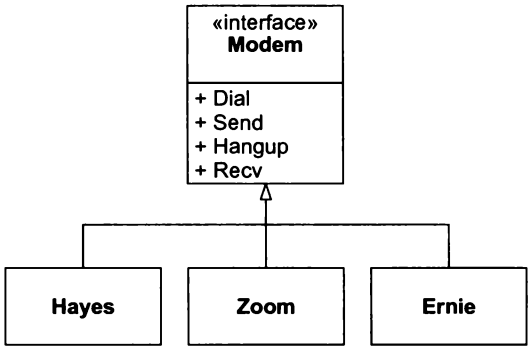


Figure 28-1 Modern Hierarchy

How can we configure these modems for UNIX without putting the `ConfigureForUnix` method in the `Modem` interface? We can use a technique called *dual dispatch*, which is the mechanism at the heart of the VISITOR pattern.

Figure 28-2 shows the VISITOR structure and Listings 28-1 through 28-6 show the corresponding Java code. Listing 28-7 shows the test code that both verifies that the VISITOR works and demonstrates how another programmer should use it.

Listing 28-1

`Modem.java`

```
public interface Modem
{
  public void dial(String pno);
  public void hangup();
  public void send(char c);
  public char recv();
  public void accept(ModemVisitor v);
}
```

1. [GOF95], p. 331.

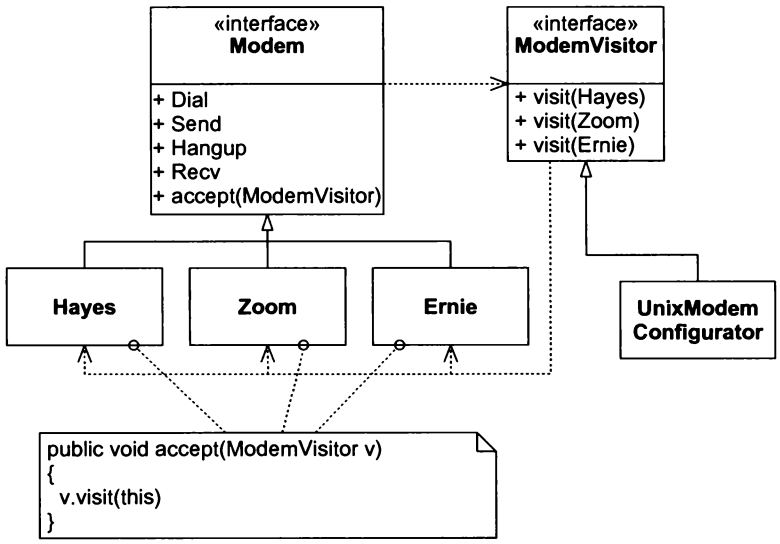


FIGURE 28-2 VISITOR

Listing 28-2

ModemVisitor.java

```
public interface ModemVisitor
{
  public void visit(HayesModem modem);
  public void visit(ZoomModem modem);
  public void visit(ErnieModem modem);
}
```

Listing 28-3

HayesModem.java

```
public class HayesModem implements Modem
{
  public void dial(String pno){}
  public void hangup(){}
  public void send(char c){}
  public char recv() {return 0;}
  public void accept(ModemVisitor v) {v.visit(this);}

  String configurationString = null;
}
```

Listing 28-4

ZoomModem.java

```
public class ZoomModem implements Modem
{
  public void dial(String pno){}
  public void hangup(){}
  public void send(char c){}
  public char recv() {return 0;}
  public void accept(ModemVisitor v) {v.visit(this);}

  int configurationValue = 0;
}
```

Listing 28-5

ErnieModem.java

```
public class ErnieModem implements Modem
{
  public void dial(String pno){}
  public void hangup(){}
  public void send(char c){}
  public char recv() {return 0;}
  public void accept(ModemVisitor v) {v.visit(this);}

  String internalPattern = null;
}
```

Listing 28-6

UnixModemConfigurator.java

```
public class UnixModemConfigurator implements ModemVisitor
{
  public void visit(HayesModem m)
  {
    m.configurationString = "&s1=4&D=3";
  }

  public void visit(ZoomModem m)
  {
    m.configurationValue = 42;
  }

  public void visit(ErnieModem m)
  {
    m.internalPattern = "C is too slow";
  }
}
```

Listing 28-7

TestModemVisitor.java

```
import junit.framework.*;
public class TestModemVisitor extends TestCase
{
  public TestModemVisitor(String name)
  {
    super(name);
  }

  private UnixModemConfigurator v;
  private HayesModem h;
  private ZoomModem z;
  private ErnieModem e;

  public void setUp()
  {
    v = new UnixModemConfigurator();
    h = new HayesModem();
    z = new ZoomModem();
```

```
    e = new ErnieModem();
  }

  public void testHayesForUnix()
  {
    h.accept(v);
    assertEquals("&s1=4&D=3", h.configurationString);
  }

  public void testZoomForUnix()
  {
    z.accept(v);
    assertEquals(42, z.configurationValue);
  }

  public void testErnieForUnix()
  {
    e.accept(v);
    assertEquals("C is too slow", e.internalPattern);
  }
}
```

Notice that there is a method in the visitor hierarchy for every derivative of the visited (`Modem`) hierarchy. This is a kind of 90° rotation—from derivatives to methods.

The test code shows that to configure a modem for UNIX, a programmer creates an instance of the `UnixModemConfigurator` class and passes it to the `accept` function of the `Modem`. The appropriate `Modem` derivative will then call `visit(this)` on `ModemVisitor`, the base class of `UnixModemConfigurator`. If that derivative is a `Hayes`, then `visit(this)` will call `public void visit(Hayes)`. This will deploy to the `public void visit(Hayes)` function in `UnixModemConfigurator`, which then configures the `Hayes` modem for `Unix`.

Having built this structure, new operating-system configuration functions can be added by adding new derivatives of `ModemVisitor` without altering the `Modem` hierarchy in any way. So the VISITOR pattern substitutes derivatives of `ModemVisitor` for methods in the `Modem` hierarchy.

This is called dual dispatch because it involves two polymorphic dispatches. The first is the `accept` function. This dispatch resolves the type of the object that `accept` is called upon. The second dispatch is the `visit` method which resolves to the particular function to be executed. These two dispatches give VISITOR very fast execution speed.

VISITOR Is Like a Matrix

The two dispatches of VISITOR form a matrix of functions. In our modem example, one axis of the matrix is the different types of modems. The other axis is the different types of operating systems. Every cell in this matrix is filled in with a function that describes how to initialize the particular modem for the particular operating system.

ACYCLIC VISITOR

Notice that the base class of the visited (`Modem`) hierarchy depends on the base class of the visitor hierarchy (`ModemVisitor`). Notice also that the base class of the visitor hierarchy has a function for each derivative of the visited hierarchy. Thus, there is a cycle of dependencies that ties all the visited derivatives (all the Modems) together. This makes it very difficult to compile the visitor structure incrementally or to add new derivatives to the visited hierarchy.

The VISITOR works very well in programs where the hierarchy to be modified does not need new derivatives very often. If `Hayes`, `Zoom`, and `Ernie` were the only `Modem` derivatives that were likely to be needed, or if the incidence of new `Modem` derivatives was expected to be infrequent, then the VISITOR would be very appropriate.

On the other hand, if the visited hierarchy is highly volatile, such that many new derivatives will need to be created, then the `Visitor` base class (e.g., `ModemVisitor`) will have to be modified and recompiled along with all its derivatives every time a new derivative is added to the visited hierarchy. In C++, the situation is even worse. The entire visited hierarchy must be recompiled and redeployed whenever any new derivative is added.

To solve these problems, a variation known as ACYCLIC VISITOR can be used.[2] (See Figure 28-3.) This variation breaks the dependency cycle by making the `Visitor` base class (`ModemVisitor`) degenerate.[3] The lack of any methods in this class means that it does not depend on the derivatives of the visited hierarchy.

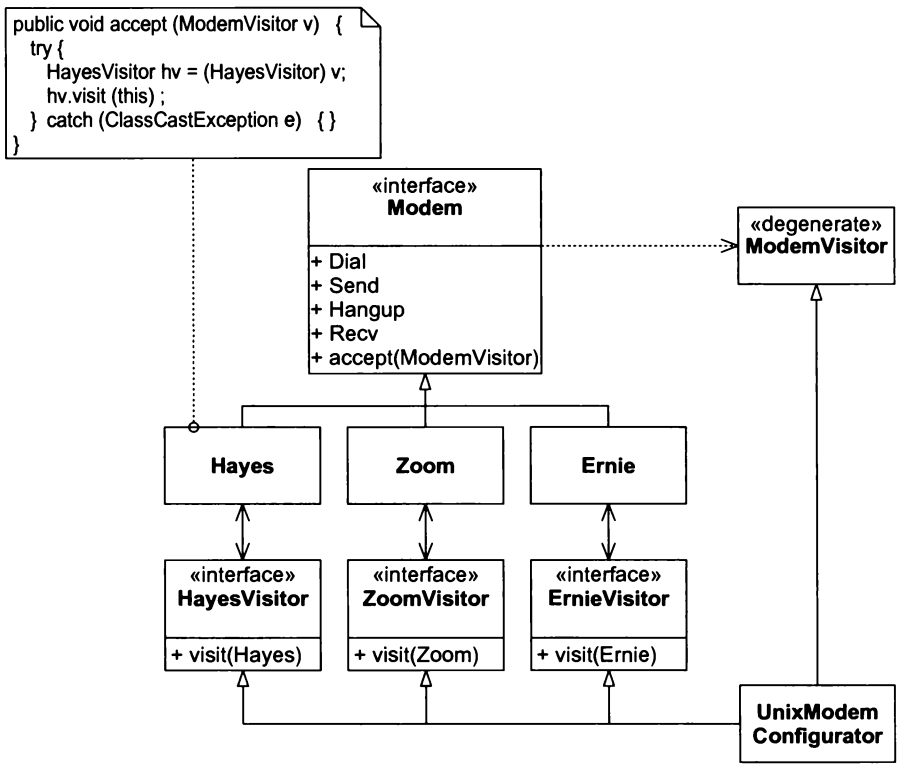


FIGURE 28-3 ACYCLIC VISITOR

The visitor derivatives also derive from visitor interfaces. There is one visitor interface for each derivative of the visited hierarchy. This is a 180° rotation from derivatives to interfaces. The `accept` functions in the visited derivatives cast the `Visitor` base class[4] to the appropriate visitor interface. If the cast succeeds, the method invokes the appropriate `visit` function. Listings 28-8 through 28-16 show the code.

2. [PLOPD3], p. 93.
3. A degenerate class is one that has no methods at all. In C++, it would have a pure virtual destructor. In Java, such classes are called "Marker Interfaces."
4. In C++, we use `dynamic_cast`.

Listing 28-8

Modem.java

```
public interface Modem
{
  public void dial(String pno);
  public void hangup();
  public void send(char c);
  public char recv();
  public void accept(ModemVisitor v);
}
```

Listing 28-9

ModemVisitor.java

```
public interface ModemVisitor
{
}
```

Listing 28-10

ErnieModemVisitor.java

```
public interface ErnieModemVisitor
{
  public void visit(ErnieModem m);
}
```

Listing 28-11

HayesModemVisitor.java

```
public interface HayesModemVisitor
{
  public void visit(HayesModem m);
}
```

Listing 28-12

ZoomModemVisitor.java

```
public interface ZoomModemVisitor
{
  public void visit(ZoomModem m);
}
```

Listing 28-13

ErnieModem.java

```
public class ErnieModem implements Modem
{
  public void dial(String pno){}
  public void hangup(){}
  public void send(char c){}
  public char recv() {return 0;}
  public void accept(ModemVisitor v)
  {
    try
    {
      ErnieModemVisitor ev = (ErnieModemVisitor)v;
```

```
      ev.visit(this);
    }
    catch (ClassCastException e)
    {
    }
  }

  String internalPattern = null;
}
```

Listing 28-14

HayesModem.java

```
public class HayesModem implements Modem
{
  public void dial(String pno){}
  public void hangup(){}
  public void send(char c){}
  public char recv() {return 0;}
  public void accept(ModemVisitor v)
  {
    try
    {
      HayesModemVisitor hv = (HayesModemVisitor)v;
      hv.visit(this);
    }
    catch (ClassCastException e)
    {
    }
  }

  String configurationString = null;
}
```

Listing 28-15

ZoomModem.java

```
public class ZoomModem implements Modem
{
  public void dial(String pno){}
  public void hangup(){}
  public void send(char c){}
  public char recv() {return 0;}
  public void accept(ModemVisitor v)
  {
    try
    {
      ZoomModemVisitor zv = (ZoomModemVisitor)v;
      zv.visit(this);
    }
    catch(ClassCastException e)
    {
    }
  }

  int configurationValue = 0;
}
```

Listing 28-16

TestModemVisitor.java

```
import junit.framework.*;
public class TestModemVisitor extends TestCase
{
  public TestModemVisitor(String name)
  {
    super(name);
  }

  private UnixModemConfigurator v;
  private HayesModem h;
  private ZoomModem z;
  private ErnieModem e;

  public void setUp()
  {
    v = new UnixModemConfigurator();
    h = new HayesModem();
    z = new ZoomModem();
    e = new ErnieModem();
  }

  public void testHayesForUnix()
  {
    h.accept(v);
    assertEquals("&s1=4&D=3", h.configurationString);
  }

  public void testZoomForUnix()
  {
    z.accept(v);
    assertEquals(42, z.configurationValue);
  }

  public void testErnieForUnix()
  {
    e.accept(v);
    assertEquals("C is too slow", e.internalPattern);
  }
}
```

This breaks the dependency cycle and makes it easier to add visited derivatives and to do incremental compilations. Unfortunately, it also makes the solution much more complex. Worse still, the timing of the cast can depend on the width and breadth of the visited hierarchy and is therefore hard to characterize.

For hard, real-time systems the large and unpredictable execution time of the cast may make the ACYCLIC VISITOR inappropriate. For other systems, the complexity of the pattern may disqualify it. But for those systems in which the visited hierarchy is volatile, and incremental compilation is important, this pattern can be a good option.